In chapter 3 our case study addressed identifying tech debt trends using PTS data. It's important to use this data to find when poor maintainability is starting to creep into your software products because it's easy to tie to the bottom line. If you're tracking how the speed of delivery slows down as a result of tech debt, it's usually not that tough to get buy-in from project sponsors to clean it up to ensure you can keep delivering at your best rate instead of constantly slowing down. You can add the data-tracking maintainability into your tech debt impact analysis to break it down into small enough chunks for your development team to take action on, and you can use the PTS data as an indicator to show your sponsors how it affects their project. You'll see more on this in chapter 9.

Static analysis is awesome and every development team should use it. But like everything else, static code analysis alone can't give you the complete picture. Some problems with using static code analysis alone are these:

- Even if the code is easy to update and has great tests, that doesn't make it easy to deploy.
- There could be integration issues with the larger systems that you can't see from static analysis.
- This doesn't check the behavior of the application.

As with the rest of your maintainability metrics, static code analysis is a great tool to use to give you actionable insight into your codebase but doesn't stand alone as the de facto measurement for maintainability.

What static analysis alone doesn't give you is the correlation between the textbook quality of your code and its actual performance, or how it relates to the productivity of your team.

8.3.5. Adding more PTS data

Adding PTS data will help round out the picture to show how all of these metrics affect how your team is delivering. We covered how to collect these metrics and their importance in previous chapters, so here I'll highlight how they tie back into indicating how maintainable your codebase is: